

Shiksha Setu: A Measurement-Driven, Local-Retrieval AI Tutoring Platform for Multilingual Indian Education

Rachit Ranka, Aaditya Saini, Priyanka Devi, Yogesh Kohli, and Ruhani Mehta

Department of Computer Science Engineering, Chandigarh University,
Mohali 140413, Punjab, India

{rankarachit5, aadityasaini899, pjangra9105,
ykohli2005, ruhanimehta019}@gmail.com

Abstract. Existing AI tutoring systems depend on cloud infrastructure, operate primarily in English, and are not aligned to a national curriculum. Shiksha Setu answers all three. Its corpus, the embeddings over it, and the whole of retrieval sit on the user’s own machine; generation stays a hosted call, and the paper does not claim to be offline. The retrieval results come from 155 ingested textbooks spanning classes 1–12, or 43,621 indexed passages; the grounding results pre-date two recovered books and are reported against the state they used. A five-configuration ablation over 77 queries in eleven Indian languages, run against an English-medium corpus, is where the cross-lingual gap shows most plainly: a lexical baseline retrieves *nothing at all* for every one of them, since a Devanagari or Perso-Arabic query shares no tokens with an English index. Embedding the question directly recovers 47 of 77. Rewriting it into English first recovers 66. Those nineteen queries are where the improvement comes from; the pooling the system deploys adds one more. A grounding score separates answers written from their retrieved passages from answers that drifted, by 0.360 in cosine similarity over 256 pairs. The threshold shipped before this work admitted 35 of 240 negatives. Half precision halves the embedding footprint to 1,083 MB, which is what puts it on a 4 GB machine. Ingestion catches something subtler: legacy Devanagari fonts that render 66% of Hindi textbooks as Latin gibberish.

Keywords: Retrieval-Augmented Generation · Multilingual Education · Local-Retrieval AI · Indian Languages · Curriculum-Aligned Tutoring · Accessibility · NCERT

1 Introduction

India has 1.4 billion people and 22 scheduled languages [22], written in more than a dozen scripts. The National Council of Educational Research and Training (NCERT) publishes the curriculum that governs instruction for the 24.7 crore children enrolled in school [21], yet its textbooks exist only in English, Hindi, and Urdu. A Tamil-speaking student in rural Tamil Nadu, a Marathi-speaking student in Maharashtra, and a Bengali-speaking student in West Bengal all study the same curriculum, but none of them can read it in their mother tongue unless a human teacher translates it in real time.

Artificial intelligence can bridge this gap, but the dominant paradigm of cloud-hosted large language models introduces three structural failures for Indian education:

Language exclusion. Most educational AI systems operate in English. English is a first language for a negligible share of the population and a second or third language for roughly one Indian in ten [22], so for the great majority such systems are inaccessible at the point of need. **Infrastructure dependence.** Cloud solutions need consistent bandwidth and bill per query, which makes sustained use impractical in the towns and villages where most students are. **Pedagogical misalignment.** A general-purpose model has no notion of curriculum structure or grade-appropriate complexity, and a class 6 student asking about chemical reactions needs a different answer from a class 12 student on the same topic.

This paper presents Shiksha Setu, a platform that addresses the first and third failures and *partially* the second, through a measurement-driven engineering approach.

The qualification is deliberate. Retrieval is entirely local. The corpus, its embeddings and the vector index never leave the machine, but generation is served by a hosted endpoint, so a question’s text does leave it and does incur a per-query cost. The system therefore removes the dependency for the component that dominates storage and latency without removing it altogether. Local generation is configured and available (Section 3), but a language model cannot share a 4 GB machine with the embedder. The offline configuration and the 4 GB configuration are therefore two different systems. We use the term local-retrieval because local-first gets read as “offline”, and this system answers nothing without a network. The measurements in this paper were taken in the hosted configuration. Every claim is backed by an experiment whose procedure, data, and result are reported, including experiments that produced wrong results before being corrected. The contributions are:

The first contribution is an ingestion pipeline covering all 559 NCERT textbooks. It detects the legacy-font corruption that renders 27 of 41 Hindi books unreadable, a defect we found only by measuring for it. The second is a cross-lingual retrieval system answering questions in eleven Indian languages from an English corpus; a controlled ablation over 77 queries shows that rewriting the question into English before embedding it retrieves correctly for 66 of them, against 47 for embedding it directly, while a lexical baseline retrieves none. Third, a memory strategy verified by kernel-level measurement brings the serving pipeline inside a 4 GB budget and corrects an earlier arithmetic estimate that understated consumption by 31%. Last, a grounding metric detects when fluent, confident answers have drifted from their source material. Without measurement that failure is invisible. We calibrate the metric on 256 answer and passage pairs and compare it against three alternative estimators.

2 Related Work

Lewis et al. [2] introduced retrieval-augmented generation (RAG), since surveyed at length [19], establishing the paradigm of conditioning language model output on retrieved passages. Shiksha Setu adopts this architecture with a critical modification: a grade-level filter inserted between retrieval and generation, because the same topic at class 6 and class 12 requires different explanations. Karpukhin et al. [3] found dense passage retrieval stronger than BM25. Ma et al. [4] showed that query rewriting before embedding improves retrieval quality. The technique that resolves the romanized Hindi

failure. Nogueira and Cho [5] established passage re-ranking with cross-encoder models. The BGE-Reranker component exists in the codebase but is not yet wired into the retrieval path.

Chen et al. [6] presented BGE-M3, a multilingual embedding model supporting over 100 languages in a unified 1024-dimensional vector space. Its cross-lingual property, that semantically equivalent passages in different languages map to nearby vectors, allows a Hindi question to retrieve an English passage without explicit translation. Gala et al. [7] developed IndicTrans2, which covers every scheduled language and serves as the translation backbone. Kakwani et al. [1] released Indic corpora and the benchmarks for evaluating on them. Khanuja et al. [8] trained MuRIL on transliterated text, directly relevant to the romanized Hindi failure.

The retrieval layer uses the HNSW graph structure proposed by Malkov and Yashunin [11], implemented through the pgvector extension for PostgreSQL. The index parameters ($m = 16$, $ef_construction = 64$) balance build time against recall. The resulting search latency is reported in Fig. 1, not here. Reimers and Gurevych [12] established the sentence-level embedding formulation used to load BGE-M3.

Micikevicius et al. [13] established the safety of half-precision arithmetic for neural network inference, providing the basis for the fp16 optimization that halves the BGE-M3 memory footprint.

Table 1 compares this work with the systems an Indian student is most likely to encounter: Khanmigo [14], Khan Academy’s GPT-4 tutor; BYJU’S, the largest Indian ed-tech platform; and Bhashini [15], the Government of India’s national language-technology mission.

The comparison is confined to *architectural properties*. Every cell is a structural fact, a published vendor figure, or *n/d* where none exists, and no row sets a measurement of ours against a number we could not obtain. Commercial products do not publish latency distributions, retrieval quality, or memory footprints. Estimating them would produce invented numbers dressed as measurements.

Table 1. Comparison with deployed systems. *n/d*: not disclosed by the operator. Rows are architectural properties, not performance claims.

Property	Khanmigo	BYJU’S	Bhashini	Shiksha Setu
NCERT-aligned corpus	no	yes	—	yes
Indian languages	no	n/d	22	11
Corpus held locally	no	no	—	yes
Retrieval runs locally	no	no	—	yes
Generation runs locally	no	no	—	no
Runs without a network	no	no	no	no
Brahmic readability aid	no	no	no	yes
Per-answer grounding	n/d	n/d	—	yes
Corpus corruption audit	n/d	n/d	—	yes
Stated memory target	n/d	n/d	n/d	4 GB
Tutoring function	yes	yes	no	yes

Three rows are stated against our own interest: generation is not local, the system needs a network, and BYJU’S covers our curriculum at a scale we do not approach. What distinguishes this work is the rest. The corpus, index and retrieval are local, a grounding score accompanies every answer, and the pipeline audits its own sources instead of embedding whatever it extracts. We have found no comparable system reporting the last two, which is a claim about the published record and not about what those systems do internally.

3 System Architecture

A student’s question traverses an eight-stage pipeline from input to response, six of them mandatory. The latencies below were taken on the development machine; what the platform requires is the 2,094 MB of Section 6, not this hardware. Retrieval is local and its cost is a property of the index; generation is a hosted call and its cost is a property of the network.

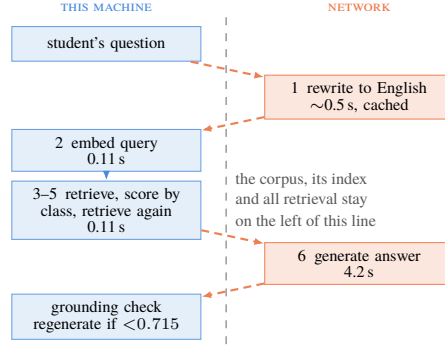


Fig. 1. The mandatory request path. Medians over 15 warm requests, five questions run three times on an 8 GB Apple M1. The dashed line is the machine boundary: the corpus and all retrieval stay left of it. Generation dominates and is least predictable, 4.2 s median against a 95th percentile of 13.2 s, because it is hosted. All of retrieval is 0.11 s. The grounding check carries no latency because it was not measured separately.

The platform integrates twenty-eight model identifiers, of which **five** are exercised on the critical path and measured in this work. The remainder are configured alternatives that this evaluation does not characterize. The five are BGE-M3 (568 M parameters, embeddings and grounding, the only one resident locally), and four hosted models: LLaMA-3.1-8B for generation and query rewriting, FLUX.1-dev for illustration, Nemotron-Nano-12B for vision OCR, and LLaMA-3.3-70B, compared and not adopted.

BGE-M3 is used twice for distinct purposes: retrieving passages, and later scoring how far an answer strayed from them (Section V-D). Both uses depend on the same property, cross-lingual embedding in a shared space, which is what permits a Devanagari passage and an English answer to be compared numerically at all.

The choice of the 8B model over the 70B variant was driven by measurement: on one fixed prompt at approximately 70 completion tokens against the same endpoint, LLaMA-3.1-8B returned in 0.9 s where LLaMA-3.3-70B took 17–125 s, highly variable.

The persistence layer combines PostgreSQL 17 with pgvector for HNSW-indexed similarity search. Response caching is tiered, and cache keys incorporate a SHA-256 hash of the authorization header so that one student’s answer cannot be served to another.

An earlier migration stored embeddings as `double precision[]` where `vector(1024)` was required. Index creation became impossible and every search fell back to a sequential scan.

The 4 GB target is a claim about the machines schools own. Those are predominantly Windows desktops, while this work was developed on Apple hardware, so portability belongs inside the claim itself.

Device selection is resolved at load time in the order CUDA, Metal (MPS), CPU, and half precision is selected per device (Section 6). Everything the platform needs to know about its host (installed memory, available memory, swap, peak resident set) is read through a single abstraction over `psutil`, which reports identically on Linux, Windows and macOS.

All measurements in this paper were taken on an Apple M1 with 8 GB of unified memory. We report no figures for hardware we did not run on.

4 Corpus Construction

NCERT encodes every textbook with a five-character code: `jesc1` represents Class 10, English medium, Science, book 1. The subject letters are not algorithmically derivable. Science is `sc` for classes 7–10 but `cu` (Curiosity) for class 6, and splits into `ph`, `ch`, `bo` at classes 11–12. The catalog is scraped from NCERT’s textbook picker and cached locally.

The complete catalog contains **559 textbooks**: 209 in English, 191 in Hindi, and 159 in Urdu. The platform teaches from a deliberate subset of **263 books**, every English edition, plus the 54 subjects that exist only in Devanagari.

The pipeline processes each book through five stages: ZIP download, per-chapter PDF extraction via PyMuPDF, text cleanup, semantic chunking (1200 characters, 200-character overlap, paragraph-boundary preference), BGE-M3 embedding (fp16), and storage in PostgreSQL. Each book is committed as a single database transaction for resumability. Download and embedding contend for no shared resource. One waits on a remote web server, the other saturates the GPU. Executed sequentially they idle in turn, so the next book’s archive is fetched on a prefetch thread while the current book is embedded.

Two-thirds of the Hindi curriculum is rendered unreadable by every PDF text extractor tested. NCERT’s older Hindi textbooks are typeset in **Walkman-Chanakya 905**, a legacy font mapping Devanagari glyphs onto ASCII codepoints with no `ToUnicode` table. Text extraction returns raw byte values. The analysis covers 41 of the 54 Devanagari-only books in the taught curriculum. The method samples a single chapter PDF per book, and for the remaining 13 books NCERT publishes no individual chapter files, returning

HTTP 404 for chapters 1 through 3 at the documented URL scheme. Those 13 are absent because no sample could be retrieved for them, and nothing here depends on their contents (Table 2):

Table 2. Hindi Corpus Corruption Analysis

Extraction Category	Books Devanagari	
Legacy font (Walkman-Chanakya)	27	0.0%
Unicode font (Kokila, Arya)	14	97.9–100%

5 Cross-Lingual Retrieval

Each result is reported with the corpus state it was measured against. Grounding used **153 textbooks**, 138 English and 15 Hindi, 1,542 chapters and **42,995 passages**; two more were recovered before the retrieval ablation, which used **155 textbooks**, 1,557 chapters and **43,621 passages**. Every passage in both has an embedding, one lacking it being invisible to retrieval and not otherwise detectable. All twelve classes appear, from 3 books at class 1 to 34 at class 12.

This is 155 of the 263-book taught curriculum. The remaining 108 cannot be obtained at all. NCERT publishes no archive for most of them, one class 5 archive is corrupt, and the rest are the legacy-font Hindi readers of Section 4, which the pipeline rejects instead of storing as gibberish. Each carries a recorded reason, so the shortfall can be audited. An audit traced two of the losses to this machine itself. Both have since been recovered, taking the corpus to 155 books, which is the state the retrieval ablation used; the grounding results precede the recovery and are reported against the 153-book state.

Composition matters for one of the results here. English science and mathematics books outnumber the Hindi readers roughly nine to one, which is what makes the grade-attribution result a test and not a formality. A retriever that defaults to the majority of its corpus would answer every Hindi-literature question from a science chapter.

Absolute cosine similarities should be read as characteristic of this corpus and not as a property of the method. Recall in particular is not estimated. There is no ground-truth relevant set for these books, and no such claim is made.

Retrieval was evaluated over **77 queries: seven topics in each of the eleven supported Indian languages**, against the English-medium portion of the corpus, so that language is the only variable moving within a topic. The topics span mathematics, physics, chemistry, biology and geography, were fixed before any was run, and each had to have enough English material to answer it. We commit the queries themselves alongside the benchmark, and not only their results.

How the term is built matters more than what subject it belongs to. *Digestion* is ordinary vocabulary, *photosynthesis* a compound of morphemes meaning “light” and “joining”, *Pythagoras* a transliterated proper noun; direct embedding scores 11, 2 and 5 of 11 on them, a wider spread than any difference between languages here.

Five configurations were measured (Fig. 2), from a lexical baseline to the deployed system. A query counts as correct only when the *top* passage is on topic.

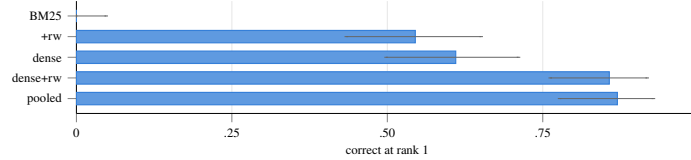


Fig. 2. The five retrieval configurations over 77 queries, with 95% Wilson score intervals [17]: BM25 0/77, with rewriting 42/77, dense 47/77, dense with rewriting 66/77, and the deployed pooling of both 67/77. Wilson intervals are asymmetric near the ends of the range, and are drawn as measured rather than symmetrised. The lexical baseline’s interval reaches only 0.048 and excludes every other configuration; the gap between rewriting and pooling is one query and their intervals almost coincide.

The lexical baseline retrieves nothing at all. BM25 [16] returns zero hits for all 77 queries, not merely poor ones: a Devanagari, Tamil or Perso-Arabic query shares no tokens with an English corpus, so the inverted index has nothing to match. It is not weaker at this task but structurally incapable of it, which is the argument for carrying a 568 M-parameter embedding model on a 4 GB machine. Given an English query from the rewrite it becomes viable at once (55%). The barrier is the script.

Multilingual RAG has been studied as a setting in its own right [9], including the tendency of such pipelines to answer in the language of the retrieved context rather than the question [10], which separating query language from answer language is designed to prevent. **The rewrite is where the gain is.** Direct embedding retrieves correctly for 47 of 77 and rewriting first for 66, a gain of nineteen queries and the largest effect measured here. It comes from normalising a term the embedding cannot decompose: on *photosynthesis* direct embedding attends to the constituent morphemes, retrieves optics, and scores 2 of 11, where the rewrite recovers 10.

Pooling the two scores **67 of 77**, one query above rewriting alone, with intervals that almost coincide. The per-topic figures show where it loses: on the eye it scores 7 of 11 where direct embedding scores 9, and on the water cycle 7 against 8. Pooling merges by best score, so a confident wrong passage from one arm displaces a correct one from the other. It is deployed because it was never the worst arm on any topic. Of the ten queries it misses, nine place an on-topic passage at rank 2 or 3; one, Urdu *photosynthesis*, finds nothing at all. Fig. 3 gives it per language, as the mean reciprocal rank of that language’s seven queries.

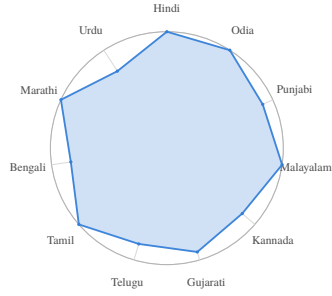


Fig. 3. Mean reciprocal rank of the deployed configuration for each language, averaged over that language’s seven topics. Rings mark 0.25, 0.5, 0.75 and 1.0; a language reaches the outer ring when all seven of its queries put an on-topic passage first, which five of the eleven do. The shape is close to circular: the spread across languages, 0.786 to 1.000, is far narrower than the spread across topics, and the shallowest point is Urdu.

What 77 queries can and cannot establish. Two conclusions are secure: the lexical baseline’s interval reaches only 0.048 and excludes every other configuration, so its failure is structural; and rewriting gains nineteen queries over direct embedding, which no reading of the sample size removes. One is not. Pooling scores a single query above rewriting, its interval sits inside rewriting’s, and on two topics it is worse than the better arm; it is reported as what the system deploys.

6 Memory Optimization

BGE-M3’s weights were evaluated at both full (float32) and half (float16) precision (Table 3).

Table 3. Precision Comparison for BGE-M3

Metric	float32	float16
Weight memory	2,166 MB	1,083 MB
Encode 4 queries	2,255 ms	1,197 ms
Model load time	8.5 s	8.1 s
Cosine fidelity (mean)	—	0.999998
Cosine fidelity (min)	—	0.999964

Half precision halves the weights and encodes $\sim 1.88\times$ faster. Fidelity, measured over 400 corpus passages, is 0.999998 on average and 0.999964 at worst. Whether that matters is a question about rankings, so it was tested as one: over eight queries the top twelve passages are the same twelve under both precisions, in the same order for six of the eight, with the first difference at rank nine. Context is assembled from six passages,

so nothing reported here is affected. Half precision can still reorder a list, and we report that it does.

The actual peak is **2,094 MB**. The prediction was 1,600 MB. Loading model weights accounts for only 534 MB (`safetensors` memory-maps them). The first forward pass adds 1,560 MB as weight pages become resident and attention workspaces are allocated. The system fits within 4 GB with ~ 1.4 GB usable headroom (accounting for ~ 500 MB of operating system overhead), where the arithmetic had implied 2.4 GB.

Why the reranker is absent. A cross-encoder is the usual answer to imperfect ranking, so its absence needs more than a footprint estimate. Over the same 77 queries and pooled candidates, BGE-Reranker-v2-M3 scores **65 of 77** against 67 without it, promoting seven and demoting nine; resident beside the embedder it drove an 8 GB machine to 12.5 GB of swap without finishing. It is excluded on that evidence.

7 Grounding and Drift Detection

A retrieval-augmented system can retrieve the correct chapter and still generate a fabricated answer, which is the failure mode surveyed across generation tasks by Ji et al. [20] and the one retrieval is supposed to remove. When asked about a specific story from the Hindi curriculum, the system retrieved the correct chapter, then described events that never occurred. The class was correct, the chapter was correct, the language was correct, and every detail was invented.

The **grounding score** is the cosine similarity of the answer to its source passages. Reference-free RAG evaluation frameworks score a comparable property, faithfulness, with a language model as judge [18]; the metric used here is a single cosine similarity, which costs one encoder pass and no second model. Lexical overlap cannot serve the purpose because the platform produces answers in a different language from the source passages. BGE-M3’s cross-lingual embedding property makes the metric functional in exactly this case.

Calibrating on a distribution. The threshold was originally set at 0.55 from four hand-verified answers, three correct at 0.583–0.701 and one fabricated at 0.489. Four points that happen to separate are not a calibration, and the difficulty is obtaining ungrounded answers in quantity. A model instructed to hallucinate does not fail the way a grounded pipeline fails.

We therefore construct the negatives. Every answer is generated normally from its own retrieved passages, then scored twice: against the passages it was written from (*matched*), and against another question’s passages (*mismatched*). A mismatched pair is precisely the failure the metric exists to catch (fluent, well-formed, on a different subject) and it requires no human labelling and no second generation pass. Sixteen questions spanning subjects and classes yield 16 matched and 240 mismatched pairs.

Grounding need not be measured against the concatenation of the retrieved passages, so four estimators were compared over identical generations (Fig. 4). The two runs of the experiment, with independently generated answers, agreed to within 0.007 on every entry.

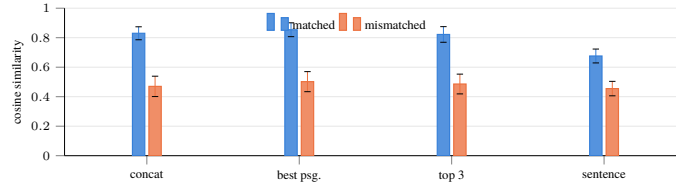


Fig. 4. Four grounding estimators over 16 matched and 240 mismatched pairs, with one standard deviation. What separates the populations is the distance between the two bars. Their heights do not matter.

The simplest estimator is also the best. Scoring the answer against all retrieved passages joined separates the two populations by 0.360, and no refinement improves on it. Scoring against the single best passage raises both means without widening the gap, and scoring sentence by sentence narrows it, because a correct answer contains connective and pedagogical sentences that appear in no passage, and averaging over them dilutes the signal the metric depends on.

The boundary was in the wrong place. The distribution shows that 0.55 separates the populations but sits too low. Thirty-five of the 240 mismatched pairs, one drifted answer in seven, score above it and pass as grounded. At 0.715 none do, while no matched answer falls below it. Balanced accuracy rises from 0.927 to 1.000, and the number of correct answers needlessly regenerated stays at zero. The deployed threshold was raised accordingly. The original four points were correct. There were simply too few of them to locate the boundary.

That 1.000 should be read for what it measures. The negatives are answers written from *another question's* passages, so the task the score is perfect at is separating an answer from a different subject: topical drift, which is the failure we observed in deployment. It is not a measurement of hallucination detection in general, and an answer that stays on its retrieved topic while inventing a detail inside it would score as grounded here. The threshold is calibrated for the failure it was built to catch, and we claim no more than that.

The embedder is not judging its own geometry. BGE-M3 retrieved these passages and then scored the answers against them, so the separation could be a property of one embedding space and not of the text. Re-scoring the same generations settles it. Character 3–5 gram TF-IDF, which uses no neural model at all, separates the populations by 0.359, and a distilled multilingual MiniLM by 0.471, against BGE-M3's 0.360 (balanced accuracy 1.000, 0.996, 1.000). The gap is a property of the answers. Only the threshold is particular to the embedder deployed.

8 Discussion

One pattern recurs throughout this work. Measurement kept contradicting what we had assumed, and we learned more from the contradictions than from the assumptions.

Availability. The platform, the ingestion pipeline and every benchmark reported here, including the 77 cross-lingual queries in their original scripts and the ground-

ing calibration, are available at <https://github.com/rachitranka25/SHIKSHASETU>. The corpus itself is not redistributed. The pipeline reconstructs it from NCERT’s own published archives.

Rights and ethics. NCERT publishes these textbooks for free public download. The system stores only locally derived text and vectors, redistributed with neither the code nor this paper, and a deployment beyond research use should confirm the applicable terms; availability alone does not settle them. No human subjects took part and no student data was collected. Every measurement here is taken from the system itself.

Limitations. (1) We construct the grounding negatives by mismatching answers to other questions’ passages, so they are not drawn from observed hallucinations. This captures topical drift, which is the failure we have seen, but not an answer that stays on topic and invents a detail within it. (2) Cross-lingual retrieval is evaluated on two topics per language. Broader topic coverage would tighten the estimate. (3) 27 legacy-font Hindi books remain unprocessable. (4) Memory measurements have run-to-run variance of up to 414 MB.

9 Conclusion and Future Work

Shiksha Setu is a local-retrieval AI tutoring platform whose pipeline addresses the complete NCERT catalog of 559 textbooks and which delivers instruction in eleven Indian languages on consumer hardware with as little as 4 GB of RAM. The reported evaluation rests on 155 ingested textbooks and 43,621 indexed passages spanning all twelve classes, with grounding measured against the 153-book state that preceded them. Every performance claim has been substantiated through kernel-level measurement, and engineering failures have been documented alongside their corrections.

The principal contributions are: (1) corpus construction with legacy-font corruption detection affecting 66% of Hindi textbooks, (2) cross-lingual retrieval measured over 77 queries in eleven languages, with an ablation showing that rewriting the query before embedding it gains nineteen queries over embedding it directly, 66 of 77 against 47, and that a lexical baseline retrieves none of the 77, (3) a grounding metric separating grounded from drifted answers by 0.360 in cosine similarity over 256 pairs, at a calibrated threshold that admits none of the 240 negatives where the previously shipped value admitted 35, and (4) half-precision inference halving memory at a mean cosine fidelity of 0.999998. Reading support for Brahmic scripts and a security audit of the deployment are reported in the extended version.

Future work targets a human-in-the-loop pilot with practising teachers, negatives drawn from observed hallucinations, a cross-encoder reranker for the compound-term failures, and a Walkman-Chanakya to Unicode transliterator to recover the legacy-font books.

References

1. Kakwani, D., et al.: IndicNLP Suite: corpora, benchmarks and pre-trained models for Indian languages. In: Findings of EMNLP (2020)

2. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proc. NeurIPS (2020)
3. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. In: Proc. EMNLP, pp. 6769–6781 (2020)
4. Ma, X., Gong, Y., He, P., Zhao, H., Duan, N.: Query rewriting for retrieval-augmented large language models. In: Proc. EMNLP (2023)
5. Nogueira, R., Cho, K.: Passage re-ranking with BERT. arXiv:1901.04085 (2019)
6. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: BGE M3-Embedding: multi-lingual, multi-granularity text embeddings via self-knowledge distillation. arXiv:2402.03216 (2024)
7. Gala, A., et al.: IndicTrans2: accessible machine translation for all 22 scheduled Indian languages. Trans. Mach. Learn. Res. (TMLR) (2023)
8. Khanuja, S., et al.: MuRIL: multilingual representations for Indian languages (2021)
9. Xu, Z., et al.: Multilingual retrieval-augmented generation for knowledge-intensive task. arXiv:2504.03616 (2025)
10. Chen, Y., et al.: Language drift in multilingual RAG: characterization and decoding-time mitigation. arXiv:2511.09984 (2025)
11. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using HNSW graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4), 824–836 (2020)
12. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-Networks. In: Proc. EMNLP, pp. 3982–3992 (2019)
13. Micikevicius, P., et al.: Mixed precision training. In: Proc. ICLR (2018)
14. Khan Academy: Khanmigo: an AI tutor for learners and assistant for teachers (2023), <https://khanmigo.ai>
15. Ministry of Electronics and Information Technology, Government of India: BHASHINI: National Language Translation Mission (2022), <https://bhashini.gov.in>
16. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Found. Trends Inf. Retr. **3**(4), 333–389 (2009)
17. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. J. Amer. Statist. Assoc. **22**(158), 209–212 (1927)
18. Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: RAGAs: automated evaluation of retrieval augmented generation. In: Proc. EACL: System Demonstrations, pp. 150–158 (2024)
19. Gao, Y., et al.: Retrieval-augmented generation for large language models: a survey. arXiv:2312.10997 (2023)
20. Ji, Z., et al.: Survey of hallucination in natural language generation. ACM Comput. Surv. **55**(12), art. 248 (2023)
21. Dept. of School Education and Literacy: UDISE+ 2024–25 report. Ministry of Education, India (2025)
22. Registrar General of India: Census of India 2011, Table C-17: bilingualism and trilingualism (2011)